

Process Management and scheduling

DAY 54
02/08/2025

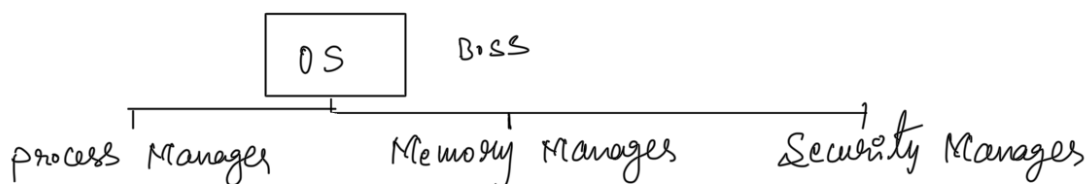
Why process concept come into picture?

Previously I have learned Basic Introduction to Operating System, I have learnt that when computer turned on Operating System or the kernel is booted after that the kernel loads the modules, The process Manager modules, Memory Manager modules and remaining modules (Basically initializes all these modules) once the modules are loaded it tells process Manager to get ready to execute other available programs or processes.

Operating System Boot → kernel Boots

↓
Initialize process Manager, Memory Manager Modules
↓
process Manager will execute next programs.

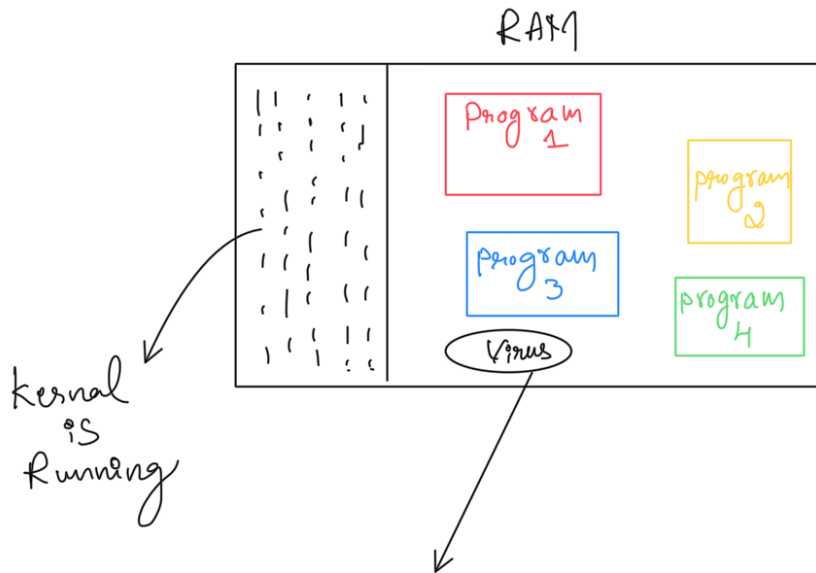
As I studied and understood that the Operating System is the Head or Boss of computer that actually protects the computer and its user.



OS is the Boss and the managers are working under the Boss (Operating System)

What is process?

Before that the application or the softwares are Run in the Binary format at OS level (softwares are in Binary language)



lets say OS is Booted and different software applications are loaded to the RAM

different programs/application are Running But How to believe that all the programs are Running are safe, like How to trust the processes?

let's say user got some suspicious mail likely a phishing attack

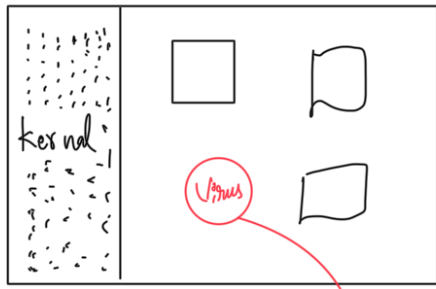
How operating system will safe guard the user information and save the computer from attack.

There is no Rule that all the programs are loaded to the RAM are safe to execute

So anyone don't wanted to their information should be leaked to the outside attacker.

So it's a responsibility of the Boss of the computer (operating system) to kill that task and make the user safe from virus.

Which the virus which is loaded to the RAM should be thrown out from RAM and process table



RAM

remove from RAM

go to process table
and free the space
where the Virus is listed
in the process table
Bcz if we remove the
Virus from the process table
then operating system will
not execute that Virus

Process table

ID	Process Name	Location of process / program file	CPU . RAM
1	P1	address	
2	P2	address	
3	P3	address	
4	Virus	address	
5	P5	address	
6	P6	address	

free the space
from process table

Delete from process table and it will
not be executed and user system will
be protected from Virus

The processor executes the process are listed in the
process table and the process's are not in the
RAM Actually, they are listed in the process table

process Table

ID	Name	location
1	P1	—
2	P2	—
3	P3	—
4	P4	—

Executes one By one

1. What is in the Process Table?

- The **process table** is a kernel data structure maintained by the Operating System.
 - Each entry is called a **Process Control Block (PCB)**.
 - PCB contains:
 - Process ID (PID)
 - Process state (running, ready, waiting, etc.)
 - CPU registers (saved state)
 - Program counter (next instruction address)
 - Memory info (page tables, base/limit registers, etc.)
 - Open files, priority, etc.
- 👉 The process table is **not** in RAM as process code, but in kernel memory (part of RAM reserved for OS). It doesn't hold full program code — only metadata and references.

Best Example is like
How Panipuri Seller
give Panipuri to the
group of People
Surrounded By Him
One By One.

Similarly the processor
executes process one
by one. (Round Robin)
There are some other algorithms
for process execution.

2. Where is the Actual Program Code?

- The **program's code and data** must be in **RAM** (at least the parts being executed), otherwise the CPU cannot fetch instructions.
- The OS uses **virtual memory + paging** to load required parts of the process into RAM from disk.
- If a page is missing, a **page fault** occurs and OS loads it from secondary storage.

3. What Does the CPU Execute?

- The **processor only executes instructions that are in RAM**.
- It does not run "directly from the process table."
- The OS **scheduler** picks a process from the **process table** (ready queue), restores its context (from PCB), and gives CPU control to that process.
- Then CPU fetches instructions from RAM (via program counter).

✓ So to answer your question clearly:

- **RAM holds the actual instructions & data** (process image).
- **Process table (PCB)** holds metadata, references, and status of processes.
- **CPU executes processes loaded in RAM**, but scheduling/selection comes from the **process table**.

👉 You can think of it like this analogy:

- **RAM = Bookshelf with actual books** (process code).
- **Process Table = Library catalog with book details** (metadata).
- **CPU = Reader who picks the next book based on catalog + librarian's (OS) scheduling.**

1. How a process (normal or virus) exists in the system

- A process has two key parts:
 - PCB (Process Control Block) → stored in the OS's process table (metadata, state, etc.)
 - Process image → actual code, data, stack, heap, etc. stored in RAM (or partly in disk if swapped)

So, the OS knows about the process via its PCB and executes it because its code is in RAM.

2. How a process is killed (normal case)

When you run `kill` in Linux, or Task Manager → End Task in Windows:

1. The OS sends a termination signal to the target process (like `SIGKILL` in Linux).
2. The CPU stops giving execution time to that process.
3. The OS deallocates all its resources:
 - Removes PCB from the process table.
 - Frees RAM occupied by the process image.
 - Closes open files, sockets, and I/O.

After that, the process is gone — both from process table and RAM.

3. Killing a Virus Process

A virus is just a malicious process — but the OS treats it like any other process.

- If you (or antivirus software) identify its PID, the OS can send a termination signal to stop it.
- Antivirus software usually hooks into the OS kernel and keeps scanning the process table + RAM to detect unknown/malicious behavior (high CPU usage, unauthorized file access, self-replication, etc.).
- Once identified:
 - The antivirus calls the OS API to kill the process (remove PCB + free RAM).
 - It may also delete the executable file from disk to prevent re-execution.

4. Why Some Viruses Are Hard to Kill

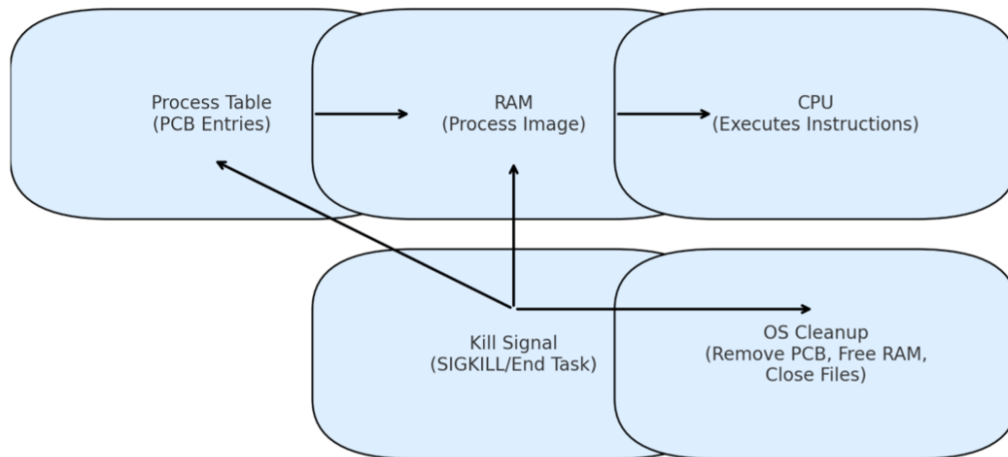
- **Privilege escalation:** Virus may run with higher privilege (kernel/root), so a normal user can't kill it.
- **Process respawning:** Virus may create a watchdog process that restarts it if killed.
- **Rootkits:** Some viruses hide themselves from the process table (modify kernel structures).

That's why antivirus + OS kernel work together, often requiring safe mode or kernel-level cleanup.

Summary:

- Killing a virus = killing its process → remove from process table + free RAM.
- Antivirus adds extra intelligence: detect + terminate + clean files/registry entries.
- The OS is always the final authority that kills the process, not the antivirus itself.

How a Virus Process is Killed in OS



This is how the virus process is killed and protects the system from attacks

Here's a diagram showing how a virus process is killed in an OS:

- The process is tracked in the Process Table (PCB).
- Its instructions are loaded in RAM and executed by the CPU.
- When you send a Kill Signal (`SIGKILL` in Linux, End Task in Windows), the OS:
 - Removes the PCB entry from the process table.
 - Frees memory used in RAM.
 - Closes any open files and releases resources.

This ensures the process is completely terminated. ✓

Imagine if process is doesn't exist in the operating system. In that case any programs (including virus) are executed and the kernel will not recognize like which program is safe which program is not. So in that case viruses can easily attack the system and take the data from the system.

So the process overcomes this issue and avoids executing virus programs and protects the system from the attackers.

And one more thing is multitasking will not be possible if process is not existed.

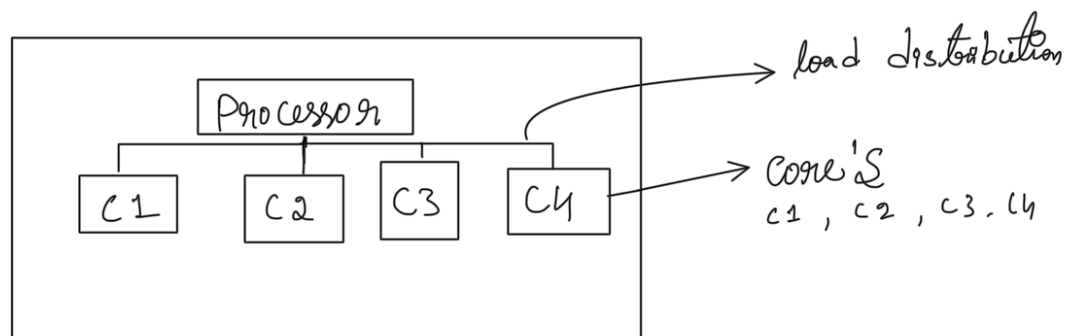
The process helps the processor for running multiple programs with the help of some process execution algorithms.

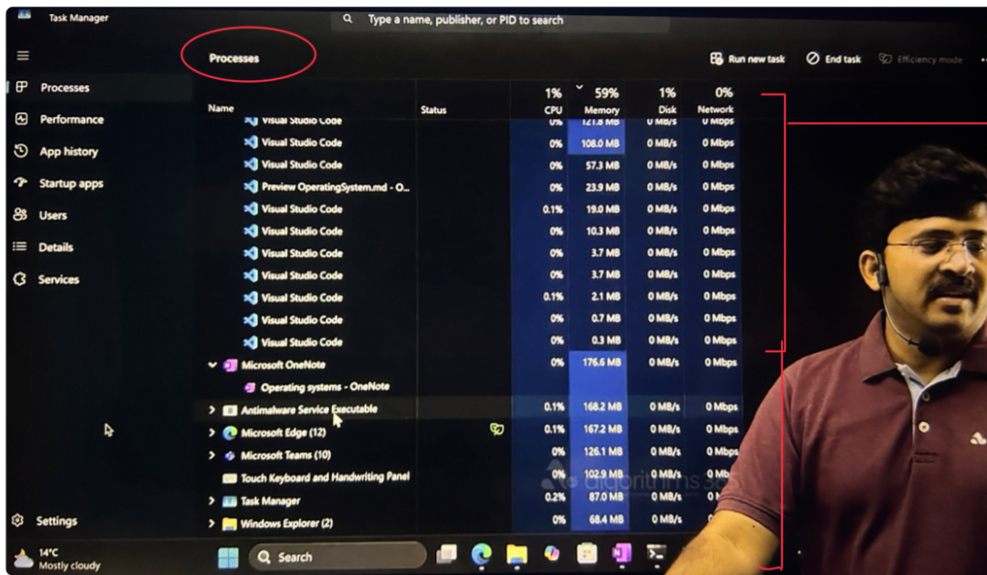
Primary Reason Process Exists

- (1) Command and Control (permission, security, risk)
- (2) Multitasking (scheduling)

Nowadays processors have multiple cores which helps for load distribution.

Multiple processors are coupled together in a chip.

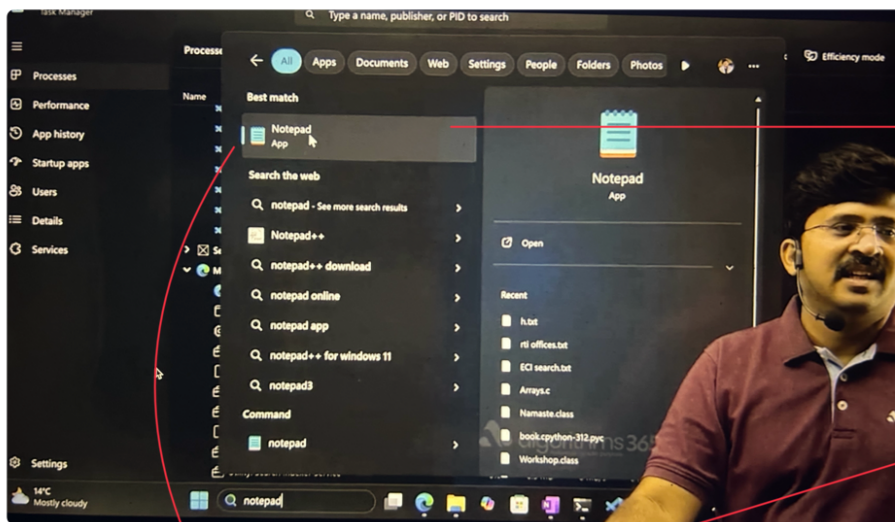




Different processes that were running

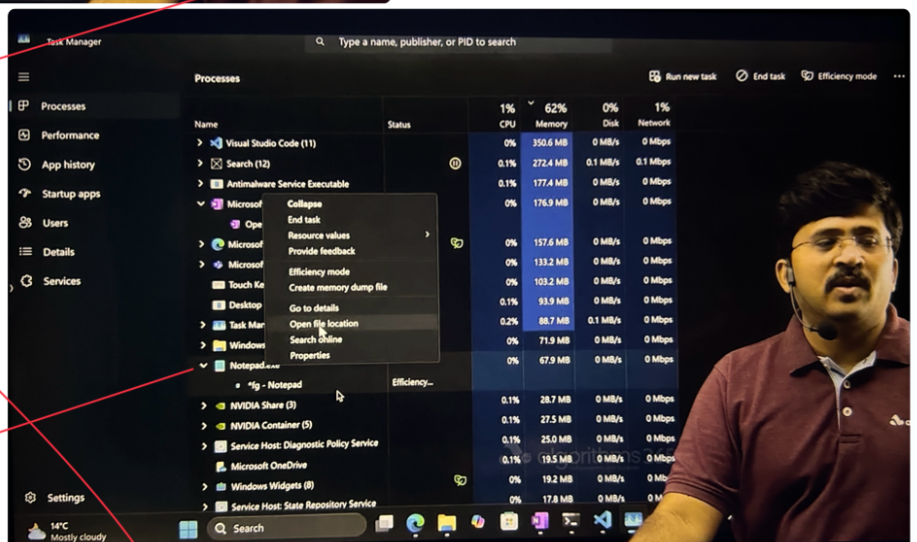
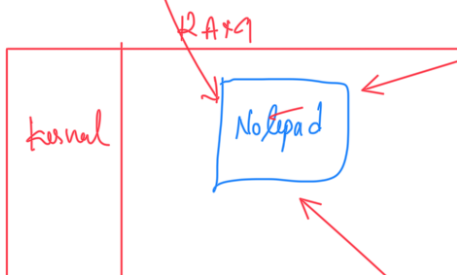
Some processors are created by user some are created by operating system itself.

let us understand more about process with example

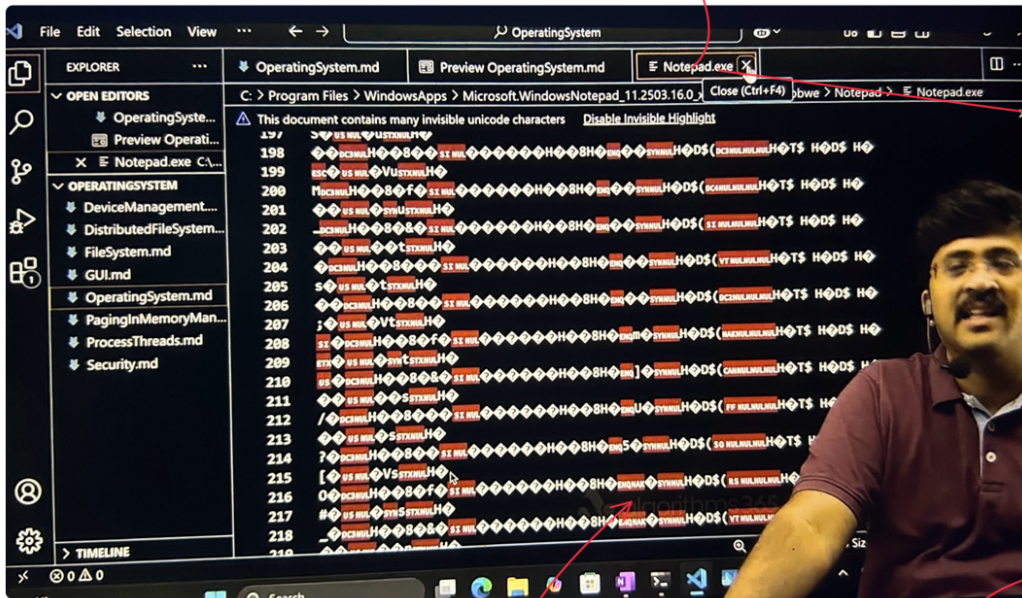


Application

When we open this Notepad application its .exe file will start executing in the ram



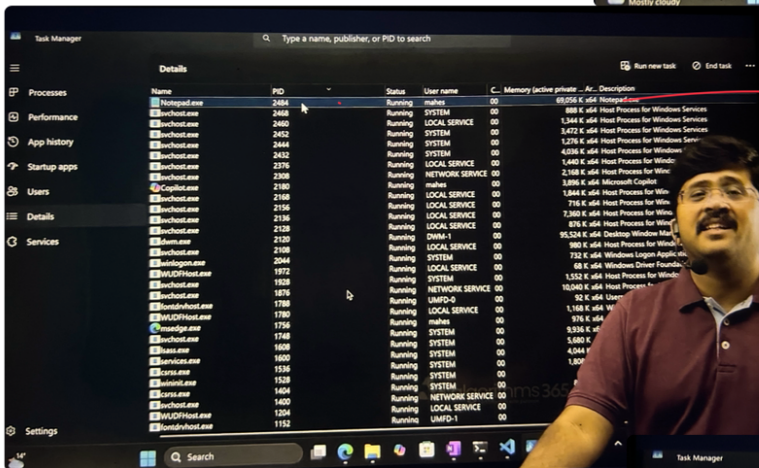
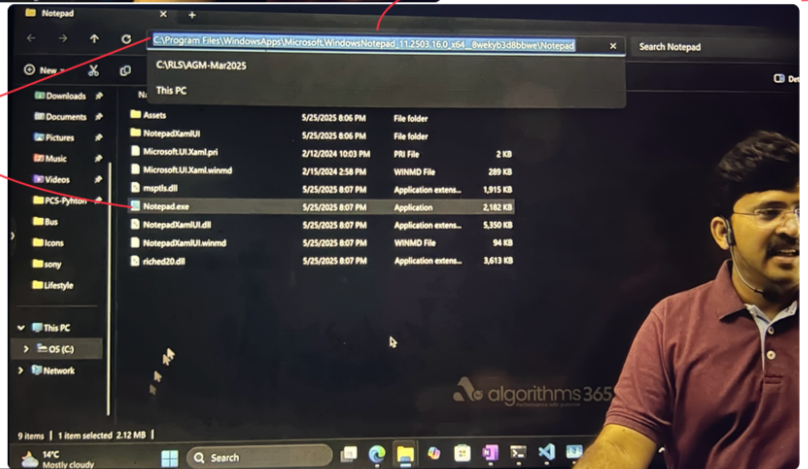
Notepad is loaded to process table



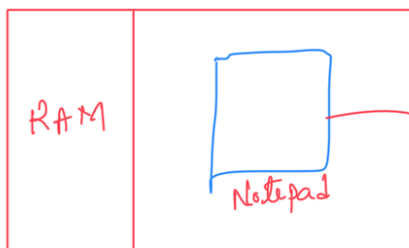
One file contains Binary Code which is used to run Notepad application.

One code is already stored somewhere in the memory

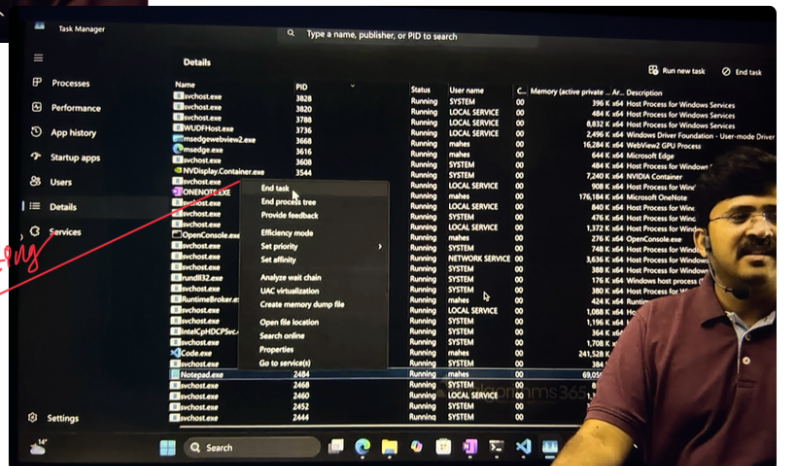
location where Notepad application stored

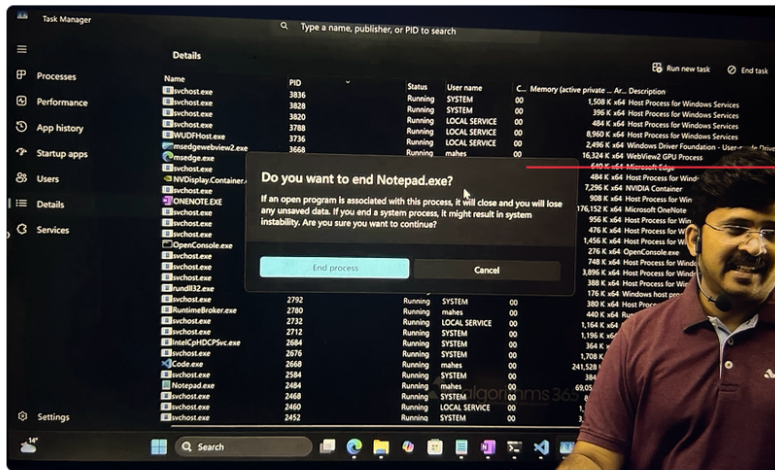


Details of the process which is running



removing the process from process table





→ Notepad Application
 is killed and the
 process stops executing
 Application will be closed
 We may lose unsaved changes
 when process is killed.

What is a process in operating system.

A process is basically a program in execution

1. Program vs Process

- A program is just a passive set of instructions (like a .exe, .class, or .py file stored on disk).
- A process is that program running in memory (active, with its own resources).

2. What a Process Contains

A process isn't just code — it has:

- **Code Section (text segment):** The actual program instructions.
- **Data Section:** Global and static variables.
- **Heap:** Dynamically allocated memory.
- **Stack:** Function calls, local variables, return addresses.
- **Registers & Program Counter:** Current state of execution.
- **Process Control Block (PCB):** Metadata like process ID (PID), state (running, waiting, etc.), scheduling info.

3. Lifecycle of a Process

- New → Ready → Running → Waiting/Blocked → Terminated
- The OS scheduler moves processes between these states.

4. Real-world Analogy

Imagine you have a recipe (program).

- The recipe book on the shelf = program on disk.
- You actually cooking the recipe in the kitchen = process in execution.

In short

Process = Program + resources
 + Execution state

Tomorrow I will learn about the memory management
 and process in more detail